

# Check-In Feature — Full Walkthrough

*Implementation guide for the leader check-in system*

Internal documentation • 19 March 2026

## CONTENTS

|                                             |                                      |
|---------------------------------------------|--------------------------------------|
| Overview                                    | Part 5 — Drill-Down Lists            |
| Roles                                       | Part 6 — Reports & History           |
| Data Storage Split                          | Part 7 — Auto-Checkout (Server-Side) |
| Part 1 — Admin Creates an Event             | Geofence Enforcement Summary         |
| Part 2 — Finding the QR Code (/checkins/qr) | Security Model Summary               |
| Part 3 — Leader Checks In                   | File Reference                       |
| Part 4 — Admin Dashboard                    |                                      |

## Overview

The check-in feature allows church **admins** to create attendance events and **leaders** to check in to those events. It is geofence-enforced, time-windowed, and supports multiple verification methods (QR, PIN, Face ID). All attendance data is stored in Firestore; member relationships and eligibility are resolved via Neo4j.

## Roles

| Role                                                                   | What they can do                                                                     |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Admin (Campus, Stream, Council, Governorship, Oversight, Denomination) | Create events, view the dashboard, manually check in members, manage event lifecycle |
| Leader (Bacenta, Governorship, Council, Stream, Campus)                | Check in to events using QR, PIN, or Face ID                                         |

## Data Storage Split

| Store                                  | What it holds                                                                      |
|----------------------------------------|------------------------------------------------------------------------------------|
| Neo4j                                  | Member identities, church hierarchy, leader/admin relationships, scope eligibility |
| Firestore <code>checkinEvents</code>   | Event config (PIN, QR secret, geofence, time window, status)                       |
| Firestore <code>checkinRecords</code>  | Individual attendance records per member per event                                 |
| Firestore <code>checkinAttempts</code> | PIN rate-limiting state (per user per event)                                       |
| Firestore <code>checkinDevices</code>  | Device fingerprint → member mapping (one device per event)                         |

## Part 1 — Admin Creates an Event

Route: `/checkins/create` (*admin only*)

The admin fills out a creation form ( `CreateCheckInEvent.tsx` ):

1. **Name & type** — e.g. "Sunday Leadership Meeting"

2. **Scope** — which church level the event covers. The form calls `GetAdminScopes`, which queries Neo4j for every church the logged-in admin holds an `IS_ADMIN_FOR` relationship on. A Campus admin sees only their Campus and below.
3. **Time window** — `startsAt`, `endsAt`, and a `gracePeriod` (in minutes). Leaders who check in after the grace period are flagged as **late** on the dashboard.
4. **Check-in methods** — one or more of `QR`, `PIN`, `FACE_ID`. At least one required.
5. **Allowed roles** — which leader types may check in: `leaderBacenta`, `leaderCouncil`, `leaderStream`, `leaderGovernorship`, `leaderCampus`.
6. **Geofence** — drawn on an interactive map ( `GeoFencePicker.tsx` ):
  - **Circle** — center point (lat/lng) + radius in metres. A "My Location" button snaps the center to the admin's current GPS.
  - **Polygon** — free-form boundary drawn by clicking vertices on the map.
7. **Auto-checkout timeout** — minutes after which inactive members are checked out. The server scheduler handles this automatically.

On submit, the `CreateCheckInEvent` mutation:

- Verifies the admin's scope authorization against Neo4j
- Generates a 6-digit **PIN** and a 32-byte random **QR secret**
- Counts eligible members via `getEligibleMembers` (all leaders within the scope hierarchy in Neo4j)
- Writes the event document to Firestore
- Returns the event with a live `qrToken`

## Part 2 — Finding the QR Code ( `/checkins/qr` )

Route: `/checkins/qr` (*all authenticated users*)

Leaders no longer need to find the admin or their dashboard to get a QR code to scan. The `/checkins/qr` page ( `CheckInQRPage.tsx` ) is location-aware:

1. On load, the browser requests GPS permission.
2. The coordinates are sent to the new `GetEventsInRange(latitude, longitude)` query.
3. The server fetches all `ACTIVE` events, runs each event's geofence check against the caller's position, and returns only the events the caller is physically inside.
4. A large, scannable QR code is displayed for each event in range — along with the event name, scope, type, and a live "ends in X:XX" countdown.
5. The page **auto-refreshes every 30 seconds** so the displayed token is always within the current 60-second rotation window.
6. If no events are in range, the page shows "No active events at your location" — no QR is ever displayed to someone outside the geofence.

This means a single device mounted at the venue (or any leader's phone at the location) can act as a shared QR display without needing admin access.

## Part 3 — Leader Checks In

Route: `/checkins/checkin`

The leader opens `MemberCheckInForm.tsx`. Before any method proceeds, **three prerequisites are validated in sequence on the server**:

### Prerequisite 1 — Geofence (*always enforced, no bypass*)

The browser sends the leader's current GPS coordinates with every check-in attempt. The server validates against the event's geofence using:

- **Haversine distance** for circle fences
- **Ray-casting algorithm** for polygon fences

If the leader is outside the boundary → blocked with a message stating how many metres away they are.

### Prerequisite 2 — Time Window

The server checks `now >= startsAt && now <= endsAt`. Outside the window → blocked.

### Prerequisite 3 — Leader Role + Scope Membership

The server queries Neo4j: the authenticated member must have a `LEADS` or `IS_ADMIN_FOR` relationship to a church, and that church must sit within the event's scope hierarchy. The member's role must also be in the event's `allowedCheckInRoles` list.

---

## Check-In Methods

Once all prerequisites pass, the leader uses one of the enabled methods:

### QR Code

- The admin's dashboard (and the `/checkins/qr` page) displays a QR code that **rotates every 60 seconds**, generated using HMAC-SHA256 with a time bucket.
- The server accepts both the current bucket and the immediately preceding one, so a scan at the moment of rotation is never rejected.
- The leader scans using the in-app camera ( `react-qr-reader` ). The decoded token is submitted with the `CheckInMember` mutation.

### PIN

- The admin shares the 6-digit PIN verbally or on-screen.
- The leader types it in. Rate limiting enforced server-side:
  - 5 failed attempts within a 10-minute window → **15-minute lockout** (stored in Firestore `checkinAttempts` )
  - On success, the attempt record is cleared
- The admin can reset the PIN at any time from the dashboard → a new 6-digit PIN is generated and the old one is immediately invalidated.

### Face ID

- The leader taps "Check in with Face ID" → `SelfieCaptureModal.tsx` opens the front-facing camera.

- `face-api.js` runs **entirely in the browser**: it extracts a 128-dimension face descriptor and computes a euclidean distance score.
- The score, status ( `VERIFIED` / `FLAGGED` ), and the base64 selfie are sent to the server.
- If the score is below threshold or status is `FLAGGED` / `SKIPPED` → the server rejects the check-in.
- Borderline scores are stored as `FLAGGED` for admin review later (see Part 5).

### Device Fingerprint (*all methods*)

- `DeviceFingerprint.ts` generates a browser fingerprint using `@fingerprintjs/fingerprintjs` (canvas fallback available).
- Sent alongside every check-in attempt. The server writes `eventId_fingerprint → memberId` to Firestore `checkinDevices`.
- If a second member tries to check in from the same device in the same event → blocked. Prevents a single phone being passed between multiple people.

## Part 4 — Admin Dashboard

Route: `/checkins/event/:eventId`

`CheckInEventDashboard.tsx` polls `GetCheckInDashboard`. The server:

1. Fetches eligible members from Neo4j (all leaders in scope)
2. Fetches all check-in records from Firestore for the event
3. Cross-references into three live buckets:

| Bucket      | Condition                                               |
|-------------|---------------------------------------------------------|
| Checked In  | Has a record, <code>checkedOutAt</code> is null         |
| Checked Out | Has a record with a <code>checkedOutAt</code> timestamp |
| Defaulted   | No record exists                                        |

4. Computes stats: total expected, checked-in count, defaulted, checked-out, attendance percentage, flagged count.

The dashboard displays:

- **Live QR code** (rotates every 60s, for leaders to scan directly from the admin's screen)
- **Stats cards** — percentage ring, counts per bucket
- **Scope filter** — for a Campus-level event, the admin can drill down to a specific Stream or Council
- **Admin controls** ( `CheckInAdminControls.tsx` ):

| Control   | Effect                                                                                                 |
|-----------|--------------------------------------------------------------------------------------------------------|
| Pause     | Sets event <code>status → PAUSED</code> . Leaders get "Event is not active" on any check-in attempt.   |
| Resume    | Sets event <code>status → ACTIVE</code> .                                                              |
| Extend    | Pushes <code>endsAt</code> forward, giving more time before the scheduler ends the event.              |
| Reset PIN | Generates a new 6-digit PIN. Old PIN is immediately invalid.                                           |
| End       | Sets <code>status → ENDED</code> . Server scheduler will process remaining check-ins on its next tick. |

## Part 5 — Drill-Down Lists

### Checked In — `/checkins/event/:eventId/checked-in`

`CheckedInMembersList.tsx` — searchable list. Shows name, unit, check-in time, method used, and whether they were late (checked in after the grace period).

### Defaulted — `/checkins/event/:eventId/defaulted`

`DefaultedMembersList.tsx` — same list format. Each row has a **Manual Check-In** button that opens `ManualCheckInModal.tsx` .

Manual check-in rules:

- Admin-only (role + scope authorization enforced on server)
- **Geofence is always required** — the modal acquires the admin's GPS before firing the mutation. If GPS is denied or the admin is outside the fence, the check-in is rejected with an error.
- Time window still applies
- All other prerequisites (method, device fingerprint, PIN/QR/Face ID) are bypassed — the admin takes responsibility by supplying a reason

### Scope Breakdown — `/checkins/event/:eventId/scopes`

`CheckInScopeBreakdown.tsx` — breaks attendance down by child scopes. A Campus-level event shows per-Stream stats, a Stream-level event shows per-Council stats, and so on.

### Flagged Reviews — `/checkins/event/:eventId/flagged` (*admin only*)

`CheckInFlaggedReview.tsx` — shows records where face match was borderline. The admin sees:

- The captured selfie
- The match score (as a percentage)
- The member's name, unit, and check-in time

The admin can either **Verify** ( `VERIFIED` ) or keep **Flagged** ( `FLAGGED` ). The resolution is written back to Firestore with `verifiedBy: admin:<authId>` for audit purposes.

## Part 6 — Reports & History

### Reports — /checkins/reports

`CheckInReports.tsx` — lists all events the user has access to (respects scope authorization), including **ENDED** past events. Firestore retains all event and attendance records permanently — nothing is deleted when an event ends.

Each event row has two actions:

- **Expand** ( **+** ) — shows an inline summary card (scope, attendance type, grace period, status)
- **Download CSV** — fetches the full dashboard for that event and triggers a browser download

The CSV export includes all three attendance buckets with the following columns:

| Column            | Populated for                                             |
|-------------------|-----------------------------------------------------------|
| Name, Role, Unit  | All rows                                                  |
| Status            | <b>Checked In</b> / <b>Checked Out</b> / <b>Defaulted</b> |
| Checked In At     | Checked In, Checked Out                                   |
| Checked Out At    | Checked Out only                                          |
| Auto Checked Out  | Checked Out only ( <b>Yes</b> / <b>No</b> )               |
| Method            | Checked In, Checked Out                                   |
| Is Late           | All rows                                                  |
| Geo Verified      | Checked In, Checked Out                                   |
| Face Match Status | Checked In, Checked Out                                   |

### History — /checkins/history

`CheckInEventHistory.tsx` — all events with status filter buttons (All / Active / Paused / Ended). Each card links directly to the event's live dashboard, which remains viewable in read-only form even after the event has ended. Also links to the Reports page for CSV downloads.

## Part 7 — Auto-Checkout (Server-Side)

Auto-checkout is handled entirely on the server. There are two triggers:

### Trigger 1 — Event Ends (Time-Based)

`checkins-scheduler.ts` runs a background job on a **60-second interval** from server startup ( `startAutoCheckoutScheduler()` called in `index.js` ).

Each tick:

1. Fetches all **ACTIVE** events from Firestore
2. Filters to those where `endsAt ≤ now`

3. Batch-updates every unchecked-out record: sets `checkedOutAt` and `autoCheckedOut: true`

4. Updates the event `status → ENDED`

No client involvement. Even if every leader closes the app, the server handles cleanup within 60 seconds of the event ending.

## Trigger 2 — Leader Leaves Geofence (Geo-Based)

While a leader is checked in, the client periodically calls the `ReportMemberLocation(eventId, latitude, longitude)` mutation with current GPS coordinates.

The server:

- Validates the coordinates against the event's geofence
- If **outside** → immediately writes `checkedOutAt` and `autoCheckedOut: true` to the member's record
- If **still inside** → no-op, returns the existing record unchanged

The client sends location pings on a timer; **the server owns the checkout decision** — no client-side logic determines whether a checkout happens.

## Geofence Enforcement Summary

Geofence is the one requirement that **cannot be bypassed** under any circumstance:

| Check-in path                         | Geofence enforced?                          |
|---------------------------------------|---------------------------------------------|
| Regular check-in (QR / PIN / Face ID) | ✓ Always                                    |
| Manual admin check-in                 | ✓ Always — admin must be at venue           |
| <code>/checkins/qr</code> page        | ✓ QR only appears if device is inside fence |
| Auto-checkout (time-based)            | N/A — server-initiated                      |
| Auto-checkout (geo-based)             | ✓ — triggers when outside fence             |



# Security Model Summary

| Mechanism               | Implementation                                                                            |
|-------------------------|-------------------------------------------------------------------------------------------|
| Role-based access       | JWT roles checked on every mutation via <code>isAuth</code> + <code>permitAdmin</code>    |
| Scope authorization     | Neo4j path query ( <code>HAS*1..6</code> ) — admins can only act on scopes they own       |
| Geofence enforcement    | Haversine (circle) / ray-casting (polygon) on every check-in path                         |
| Device fingerprinting   | <code>@fingerprintjs/fingerprintjs</code> — one device per member per event               |
| PIN rate limiting       | 5 attempts / 10 min window → 15 min lockout, tracked in Firestore                         |
| QR token expiry         | HMAC-SHA256 with 60-second rotating time bucket                                           |
| Face ID threshold       | Euclidean distance score — borderline scores flagged for admin review                     |
| Leader-only enforcement | Neo4j <code>LEADS</code> / <code>IS_ADMIN_FOR</code> relationship check on every check-in |

## File Reference

### Backend ( `api/src/resolvers/checkins/` )

| File                               | Purpose                                                                          |
|------------------------------------|----------------------------------------------------------------------------------|
| <code>checkins-resolvers.ts</code> | All GraphQL query and mutation handlers                                          |
| <code>checkins-service.ts</code>   | Business logic: eligibility, scope checks, PIN policy, device tracking, geofence |
| <code>checkins-scheduler.ts</code> | Background job: time-based auto-checkout and event lifecycle                     |
| <code>checkins-utils.ts</code>     | Helpers: PIN generation, QR token generation/validation, time window check       |
| <code>checkins-geo-utils.ts</code> | Haversine distance, circle check, polygon ray-casting                            |
| <code>checkins-types.ts</code>     | TypeScript interfaces mirroring the GraphQL schema                               |
| <code>firebase.ts</code>           | Firestore client initialisation (lazy-loaded, service account credentials)       |

### Backend ( `api/src/schema/` )

| File                          | Purpose                                                                   |
|-------------------------------|---------------------------------------------------------------------------|
| <code>checkins.graphql</code> | All GraphQL types, enums, queries, and mutations for the checkins feature |

| File                                      | Purpose                                                                            |
|-------------------------------------------|------------------------------------------------------------------------------------|
| <a href="#">checkinsRoutes.ts</a>         | Route definitions and role guards for all checkins pages                           |
| <a href="#">checkinsQueries.ts</a>        | All Apollo GQL queries and mutations used by the frontend                          |
| <a href="#">CheckInsChurchSelect.tsx</a>  | Entry landing page — church selection                                              |
| <a href="#">CheckInEventsByChurch.tsx</a> | Lists events for a given church with status filters                                |
| <a href="#">CreateCheckInEvent.tsx</a>    | Admin form to create a new check-in event                                          |
| <a href="#">CheckInQRPage.tsx</a>         | Location-aware public QR display — shows QR for events at current position         |
| <a href="#">MemberCheckInForm.tsx</a>     | Leader check-in form (GPS, QR scan, PIN entry, Face ID)                            |
| <a href="#">CheckInEventDashboard.tsx</a> | Live dashboard: stats, QR display, scope filter, admin controls                    |
| <a href="#">CheckInAdminControls.tsx</a>  | Pause / resume / extend / reset PIN / end event                                    |
| <a href="#">CheckInScopeBreakdown.tsx</a> | Attendance breakdown by child scope                                                |
| <a href="#">CheckedInMembersList.tsx</a>  | Searchable list of checked-in members                                              |
| <a href="#">DefaultedMembersList.tsx</a>  | Searchable list of defaulted members with manual check-in trigger                  |
| <a href="#">ManualCheckInModal.tsx</a>    | Admin modal for manual check-in (geofence-enforced)                                |
| <a href="#">CheckInFlaggedReview.tsx</a>  | Admin face-match review panel (verify or keep flagged)                             |
| <a href="#">CheckInReports.tsx</a>        | Event list with CSV export                                                         |
| <a href="#">CheckInEventHistory.tsx</a>   | Past events summary                                                                |
| <a href="#">GeoFencePicker.tsx</a>        | Interactive map component for drawing circle or polygon geofences                  |
| <a href="#">SelfieCaptureModal.tsx</a>    | Front-camera selfie capture modal for Face ID                                      |
| <a href="#">DeviceFingerprint.ts</a>      | Browser fingerprint generation ( <a href="#">@fingerprintjs/fingerprintjs</a> )    |
| <a href="#">FaceMatchService.ts</a>       | Client-side face descriptor extraction and scoring ( <a href="#">face-api.js</a> ) |